

Session Management with SessionAuth

Remembering Users Across Requests

The Problem: HTTP Is Stateless

```
User visits page 1: "Hi, I'm John"  
User visits page 2: "Who are you again?"
```

Each request is independent — the server forgets everything in between.

- **Sessions** solve this by giving the server a place to remember a visitor.

How Sessions Work

Think of a session like an amusement-park wristband.

- The park stores your information, and the band carries only your number.
- PHP creates the session ID and gives it to the browser in a cookie called `PHPSESSID`.
- The browser sends this ID with later requests, so PHP can find the saved information.

```
session_start();           // must run first
$_SESSION['username'] = 'john'; // server stores this
```

1. `session_start()` creates (or resumes) a session and a unique ID.
2. PHP sends that ID to the browser as a cookie (`PHPSESSID`).
3. The browser returns the cookie on every later request.
4. PHP uses the cookie to reload the same `$_SESSION` data.

A Minimal Login/Logout Pattern

```
// login.php
session_start();
if ($_POST['username'] === 'john' && $_POST['password'] === 'Secret123') {
    $_SESSION['user_id'] = 1;
    $_SESSION['username'] = 'john';
    $_SESSION['logged_in'] = true;
}
```

```
// logout.php
session_start();
session_destroy(); // clears server-side session data only
```

- `session_destroy()` deletes the session data stored on the server, but it does not clear in-memory `$_SESSION` for the current request, nor does it delete the browser's `PHPSESSID`

The password check above compares plain text () directly.

- That is enough to demonstrate a session, but it is not secure authentication.
- Next lecture replaces this with real password hashing.

How the Cookie is Read

- `login.php` saves `$_SESSION['username']` `= 'john'` after a successful login.
- PHP sends the browser a `PHPSESSID` cookie, and the browser automatically includes it when the user later visits any page on the same server, such as `dashboard.php`.

- When that page calls `session_start()`, PHP uses the cookie's ID to find John's saved session data, so `$_SESSION['username']` is available again.
- The cookie contains only the ID, not the username itself.
- Restoring the session does not automatically protect the page; the page must still check `$_SESSION['logged_in']`.

Protecting a Page

- If `$_SESSION['logged_in']` is `true`, the visitor may continue.
- Otherwise, the page redirects the visitor to the login page.

```
session_start();  
if (!($_SESSION['logged_in'] ?? false)) {  
    header('Location: login.php');  
    exit;  
}
```

The SessionAuth Class

`SessionAuth` puts common session tasks in one reusable class: starting the session, saving login information, checking login status, and logging out.

```

class SessionAuth {
    public function __construct() {
        if (session_status() === PHP_SESSION_NONE) {
            session_start();
        }
    }

    public function login_user($user) {
        $_SESSION['user_id'] = $user['id'];
        $_SESSION['username'] = $user['username'];
        $_SESSION['logged_in'] = true;
        session_regenerate_id(true);    // prevent session fixation
    }

    public function is_logged_in() {
        return isset($_SESSION['logged_in']) && $_SESSION['logged_in'] === true;
    }

    public function require_auth($login_url = 'login.php') {
        if (!$this->is_logged_in()) {
            header("Location: $login_url");
            exit;
        }
    }

    public function logout_user() {
        session_unset();
        session_destroy();
    }
}

```

SessionAuth: Methods and Session Safety

- `__construct()` starts the session so PHP can read and write `$_SESSION`.
- `login_user($user)` saves the user's ID and name, then marks the user as logged in.
- `is_logged_in()` checks whether the session says that a user is logged in.
- `require_auth()` redirects a visitor to the login page if they are not logged in.
- `logout_user()` removes the saved session data and logs the user out.

- The class does not check passwords; Login code must verify the password first, then call `login_user($user)`.
- `session_regenerate_id(true)` gives the user a new session ID after login and removes the old one.
- This prevents an old, exposed session ID from being reused after login. This attack is called session fixation.

On a protected page, the result is simple:

```
$session = new SessionAuth();  
$session->require_auth();
```

- If `is_logged_in()` returns `true`, `require_auth()` does nothing and the page continues normally.
- After a successful login, the visitor returns to the protected page and the check can pass.

- If it returns `false`, `require_auth()` sends the visitor to `login.php` with `header('Location: login.php')`.
- `exit` stops the current page immediately, so protected content is not shown to a visitor who is not logged in.

```
public function require_auth($login_url = 'login.php') {  
    if (!$this->is_logged_in()) {  
        header("Location: $login_url");  
        exit;  
    }  
}
```

Using SessionAuth

```
// Any protected page  
$session = new SessionAuth();  
$session->require_auth();
```

One line replaces the repeated `session_start()` + check pattern from before — and every page enforces the exact same rule.

We name the variable `$session` on purpose — later, when we meet the `Auth` class, `$auth` will mean something different: checking a password, not remembering a visitor.

Sessions vs. Cookies

	Server storage	Client storage	Best for
Session	Yes	Session ID only	Login state, sensitive data
Cookie	No	Full data	Non-sensitive preferences

Sessions work through cooperation between PHP and the browser: PHP stores the important data on the server, while the browser stores and returns the session ID that lets PHP find that data.

Hands-On

Try both versions in the repository:

- `7_Security/Basics/session/basic` — the repeated, manual pattern
- `7_Security/Basics/session/sessionauth` — the `SessionAuth` class above

- Run `php -S localhost:8080` in either folder and compare how much code you have to repeat per page.
- Open `http://localhost:8080/login.php` in a web browser and log in.

Key Takeaways

1. HTTP is stateless; **sessions** let the server remember a visitor.
2. `session_start()` must run before reading or writing `$_SESSION`.
3. Wrap the repeated login-check pattern in one class (`SessionAuth`).
4. Regenerate the session ID on login to block session fixation.
5. We use `$session` for a `SessionAuth` object — `$auth` is reserved for the `Auth` class next lecture.